

Guion de sustentación

Impacto de la disponibilidad de ChatGPT en el desarrollo de software orientado a Ciencia de Datos (2020–2023)

Jesus Alberto Nicho Rosado • Karl Willem Janampa Aparicio
Asesor: Alexander Wilder Quispe Rojas • PUCP, Lima 2026

Texto para decir en voz alta. Cada sección indica, a la derecha de su título, su tiempo objetivo y su responsable:

Expositor 1 (planteamiento: bloques 1–5) • **Expositor 2** (parte empírica: bloques 6–10) • **Ambos** (cierre). Total: 50 minutos, solo el cuerpo.

1. Apertura y motivación

(0:00–5:00) — Expositor 1

■ Portada

[0:20]

Buenos días. Agradecemos al jurado por su tiempo. Presentamos nuestra tesis sobre el impacto causal de la disponibilidad de ChatGPT en el desarrollo de software, durante el período comprendido entre 2020 y 2023.

■ Agenda

[0:30]

La exposición sigue diez bloques que van del problema y la teoría hacia los datos, el método y los resultados. Yo presentaré la primera parte —motivación, pregunta de investigación y marco teórico— y mi compañero presentará la parte empírica. Comencemos por entender por qué este problema es relevante.

■ Motivación

[2:00]

ChatGPT se lanzó el 30 de noviembre de 2022 y marcó un punto de inflexión en la asistencia con inteligencia artificial para tareas de programación. A diferencia de tecnologías previas como Copilot o Codex, su escala fue cualitativamente distinta: fue un servicio gratuito, masivo y conversacional, que alcanzó a millones de usuarios en pocas semanas.

Hay un hecho adicional que lo vuelve especialmente interesante para la investigación: su disponibilidad no fue homogénea. ChatGPT estuvo bloqueado en países como China, Rusia, Irán, Cuba y Corea del Norte, entre otros. Esa restricción geográfica, que no respondió a decisiones de los desarrolladores, genera un escenario cuasi-experimental natural, donde unos países recibieron el tratamiento y otros no.

Y la magnitud económica no es menor: el sector software se proyectaba en torno a los 600 mil millones de dólares para 2025. En un mercado de esa escala, cualquier mejora marginal de productividad tiene consecuencias agregadas relevantes. Por eso vale la pena medir este efecto con rigor.

■ ¿Por qué importa?

[2:10]

Si ChatGPT efectivamente aumenta la productividad o la entrada de nuevos desarrolladores, la respuesta tiene implicaciones prácticas en al menos tres frentes.

En política pública, define decisiones sobre subsidios al acceso, regulación de los servicios de inteligencia artificial y el manejo de las restricciones internacionales. En el ámbito empresarial, orienta la inversión en suscripciones premium, los programas de adopción y la forma de evaluar la productividad de los equipos. Y en educación, influye en los currículos de ciencia de datos, en la formación de los desarrolladores junior y en las certificaciones laborales.

El punto es que hoy estas decisiones se están tomando sin evidencia causal sólida. Sin esa evidencia, se decide a ciegas. Esa es precisamente la brecha que nuestra tesis busca llenar, y nos lleva directamente a la pregunta de investigación.

2. Pregunta e hipótesis

(5:00–8:00) — Expositor 1

■ Pregunta de investigación

[1:30]

Nuestra pregunta central es la siguiente: ¿la disponibilidad de ChatGPT causó un incremento del número de desarrolladores activos en GitHub en los países con acceso, comparado con los países donde el servicio fue restringido, entre 2020 y 2023?

Para responderla, la unidad de análisis es país por lenguaje de programación por trimestre. El tratamiento es el acceso público a ChatGPT a partir de noviembre de 2022. La variable de resultado es el número de *unique pushers* por cada 100 mil habitantes. Y el período abarca desde el primer trimestre de 2020 hasta el cuarto trimestre de 2023, es decir, dieciséis trimestres. De esta pregunta se desprenden dos hipótesis.

■ Hipótesis

[1:30]

La primera hipótesis plantea que la accesibilidad a ChatGPT se asocia positivamente con el desarrollo de software en los países tratados. La segunda es más específica: el efecto sería heterogéneo por lenguaje, más fuerte en Python y JavaScript, porque su amplia documentación técnica disponible en internet es justamente el recurso que alimentó el entrenamiento del modelo.

Detrás de estas hipótesis hay dos mecanismos teóricos. El primero es la reasignación de tareas, de Acemoglu y Restrepo: la inteligencia artificial libera tareas rutinarias, expande la frontera de proyectos rentables y atrae programadores marginales. El segundo es la reducción de los costos de búsqueda y aprendizaje, en la línea de Stigler: ChatGPT lleva a niveles cercanos a cero el costo de explorar documentación y foros. Veamos estos dos mecanismos con más detalle.

3. Marco teórico

(8:00–14:00) — Expositor 1

■ Marco teórico (I): Acemoglu y Restrepo

[1:50]

El primer pilar es la teoría de reasignación de tareas de Acemoglu y Restrepo. Según este marco, el cambio tecnológico actúa sobre el trabajo a través de dos canales simultáneos.

Por un lado, está el efecto de desplazamiento: ChatGPT automatiza tareas que antes realizaba el programador, como buscar sintaxis, redactar código repetitivo o depurar de forma rutinaria. Por otro lado, está el efecto de reinstauración: al bajar el costo marginal de producir código, proyectos que antes eran inviables se vuelven rentables, y programadores que antes no podían entrar al mercado ahora pueden hacerlo. La frontera productiva se expande.

Bajo condiciones plausibles, el efecto neto de ambos canales sobre la actividad de desarrollo es positivo. Este marco explica el qué; el segundo pilar explica por qué entran sobre todo los programadores nuevos.

■ Marco teórico (II): Stigler

[1:40]

El segundo pilar es la economía de la información de Stigler, quien modela la búsqueda como un costo que el agente paga para reducir su incertidumbre sobre las alternativas.

En el desarrollo de software, ese costo se traduce en el tiempo que el programador dedica a leer documentación oficial, a buscar en Stack Overflow y otros foros, y a experimentar con código

que falla. ChatGPT actúa como un *shock* negativo sobre ese costo, porque ofrece respuestas contextualizadas e instantáneas.

Y hay evidencia coherente con este mecanismo: Del Río-Chanona y coautores documentan una caída del 16 por ciento en la actividad de Stack Overflow tras el lanzamiento de ChatGPT, lo que indica una reorganización de los canales de búsqueda. Ahora bien, ¿cómo integramos ambos mecanismos en un modelo que podamos estimar? Ese es nuestro aporte teórico.

■ Marco teórico (III): modelo formal de adopción heterogénea [2:30]

Integramos los dos mecanismos en un modelo de tareas con decisión de entrada al mercado. La idea central es la siguiente: el desarrollador elige, en cada tarea, el método más productivo entre hacerlo por su cuenta o con ayuda de ChatGPT.

A partir de ahí, el modelo resume la ganancia de cada lenguaje en un parámetro de exposición, que llamamos θ . La exposición combina cuántas tareas del lenguaje son automatizables y cuánto ahorro aporta la herramienta en esas tareas. Y resume la capacidad de cada país para convertir esa ganancia en nuevos desarrolladores en un parámetro de absorción, que llamamos ϕ , y que depende de cuánta gente está cerca del umbral de entrada a la programación.

El resultado del modelo es la ecuación que estimamos. Lo importante es que el coeficiente del tratamiento es el producto de exposición por absorción. Al ser un producto, el efecto se concentra donde coinciden un lenguaje muy expuesto y un país con alta capacidad de absorción; si una de las dos es baja, el efecto se atenúa. Esa complementariedad es la forma precisa en que el modelo formaliza la adopción heterogénea, y es lo que sustenta nuestra segunda hipótesis. La derivación completa, paso a paso, está en el Apéndice B por si desean revisarla. Antes de pasar a los datos, veamos qué encontró la literatura previa.

4. Evidencia empírica previa (14:00–16:30) — Expositor 1

■ Evidencia: impacto económico de la IA [1:15]

La evidencia previa sobre el impacto económico de la inteligencia artificial es mixta. En algunos contextos el efecto es positivo: Brynjolfsson y coautores encuentran un aumento del 14 por ciento en la productividad de agentes de atención al cliente, concentrado en los trabajadores con menos experiencia, lo que es coherente con nuestra teoría. En otros contextos el efecto es negativo: estudios como los de Hui o Demirci documentan caídas en la demanda de trabajos automatizables. En síntesis, el signo del efecto depende del contexto. Pero hay un patrón aún más relevante para nosotros, que aparece cuando se mira por lenguaje de programación.

■ Evidencia: capacidades por lenguaje [1:15]

Tres estudios de *benchmark* —Buscemi, Chai y coautores, y Li y Krishnamachari— coinciden en que el desempeño de ChatGPT depende fuertemente del lenguaje. El modelo rinde claramente mejor en lenguajes como Python, Java o R, y muestra limitaciones en lenguajes funcionales poco comunes. Esto respalda nuestra segunda hipótesis.

Sin embargo, aquí está la brecha que motiva la tesis: ninguno de estos estudios mide el impacto causal de ChatGPT sobre el desarrollo de software real, desagregado por lenguaje y por país. Eso es lo que aportamos. Un último matiz sobre la ventana temporal antes de pasar a los datos.

5. Línea de tiempo (16:30–19:00) — Expositor 1

■ Línea de tiempo de ChatGPT [1:15]

Esta línea de tiempo ubica los hitos relevantes del período en tres carriles paralelos: en azul, la evolución de GitHub Copilot; en verde, la de ChatGPT; y en rojo, las limitaciones técnicas del modelo, con sus duraciones. El punto que quiero destacar es que, durante nuestra ventana de estudio, el producto no fue estático: fue cambiando de manera importante. Y eso tiene una implicación directa para cómo interpretamos el efecto.

■ Implicaciones para la interpretación

[1:15]

La ventana postratamiento, del cuarto trimestre de 2022 al cuarto de 2023, no fue homogénea. En los primeros meses operaba GPT-3.5, gratuito pero con alucinaciones frecuentes, una ventana de contexto reducida y un corte de entrenamiento en septiembre de 2021. A partir del segundo trimestre de 2023 aparece GPT-4, mucho más capaz, pero de pago.

Esto significa que nuestro efecto promedio mezcla dos regímenes distintos, por lo que podría no ser monotónico en el corto plazo. Lo señalamos abiertamente como una limitación de interpretación. Con el planteamiento y la teoría ya definidos, le cedo la palabra a mi compañero, que presentará cómo llevamos todo esto a los datos.

6. Datos y diseño del panel

(19:00–25:00) — Expositor 2

■ Fuente y variable de interés

[1:30]

Gracias. Nuestra fuente de datos es el GitHub Innovation Graph, una base pública mantenida por GitHub con métricas agregadas de actividad de desarrollo por país, lenguaje y trimestre.

La variable de resultado es el número de *unique pushers* por cada 100 mil habitantes. Un *unique pusher* es un desarrollador único que realizó al menos un `git push` en GitHub durante el trimestre, y lo normalizamos por población. Esta variable mide la entrada y la persistencia de programadores activos. No mide la intensidad por persona, ni la calidad del código, ni el valor agregado. Y reconocemos una limitación: países como China usan plataformas alternativas como Gitee, lo que puede subestimar la actividad en el grupo de control. Sobre esta variable construimos el panel.

■ Diseño del panel

[1:30]

El panel tiene tres dimensiones. Primero, 159 países: 130 tratados, con acceso, y 29 de control, sin acceso. Segundo, diez lenguajes de programación: C, C#, C++, Go, Java, JavaScript, PHP, Python, Ruby y TypeScript. Y tercero, dieciséis trimestres, del primero de 2020 al cuarto de 2023. En total, más de 25 mil observaciones.

Tenemos doce trimestres de pretratamiento y cuatro de tratamiento. Y excluimos tres casos: la Unión Europea, por ser un agregado y no un país; Kosovo, por no tener un código ISO procesable; y Hong Kong, por ser un valor atípico. Veamos la asignación en el mapa.

■ Mapa global de disponibilidad

[0:40]

Este mapa muestra la asignación del tratamiento. En azul están los 130 países con acceso, que forman el grupo tratado, y en gris claro los países sin acceso, que forman el control. Como se observa, buena parte de los controles son países con restricciones de internet, un punto sobre el que volveremos en las pruebas de robustez.

■ Estadísticos descriptivos clave

[1:00]

Esta tabla compara la media de *unique pushers* antes y después del tratamiento, para control y tratados. Conviene una advertencia de lectura: ambos grupos crecen a lo largo del período. Por eso, comparar solo los niveles puede llevar a conclusiones equivocadas. El efecto causal no está en

los niveles, sino en comparar las tasas de cambio entre tratados y controles, que es exactamente lo que hace nuestra metodología. Lo vemos con más claridad en las tendencias temporales. Primero, el grupo de control.

■ Tendencias temporales (control) [0:40]

La línea vertical roja marca el lanzamiento de ChatGPT, en el cuarto trimestre de 2022. Lo que quiero que observen es que, en el grupo de control, las series continúan su trayectoria sin mostrar una aceleración tras el tratamiento. Comparemos esto con los países que sí tuvieron acceso.

■ Tendencias temporales (tratado) [0:40]

Aquí, en cambio, sí se aprecia una aceleración después de la línea roja, concentrada en JavaScript, Python y TypeScript, que son precisamente los lenguajes que la teoría prioriza. Esto es todavía evidencia descriptiva; la causalidad la establecen los métodos que veremos a continuación.

7. Metodología

(25:00–33:00) — Expositor 2

■ Marco común: resultados potenciales [1:30]

Trabajamos en el marco de resultados potenciales. El parámetro que nos interesa es el efecto promedio del tratamiento sobre los tratados, el ATT, que captura cuánto cambió la variable de resultado en los países con acceso por causa del tratamiento.

Para estimarlo usamos tres estrategias complementarias: Diferencias en Diferencias, Control Sintético y Diferencias en Diferencias Sintética. Esta última, que abreviamos SDiD, es nuestro método principal. La razón de usar tres métodos con supuestos distintos es que, si todos coinciden en el resultado, la conclusión es mucho más creíble. Empecemos por el más conocido.

■ Método 1: DiD [1:30]

Diferencias en Diferencias compara el cambio antes y después del tratamiento en los países tratados, contra ese mismo cambio en los países de control. Es un método transparente, pero descansa en un supuesto fuerte: el de tendencias paralelas, es decir, que en ausencia de tratamiento ambos grupos habrían evolucionado igual.

En nuestro caso ese supuesto es cuestionable, porque el grupo de control es heterogéneo: son países con perfiles geopolíticos muy distintos. Por eso no nos quedamos solo con Diferencias en Diferencias. El Control Sintético atiende precisamente esa heterogeneidad.

■ Método 2: Control Sintético (CS) [1:30]

El Control Sintético construye un contrafactual a la medida. En lugar de comparar contra el promedio simple de los controles, construye un país tratado sintético, como un promedio ponderado de los controles, elegido para reproducir la trayectoria del tratado real antes del tratamiento.

Su ventaja es que maneja mejor la heterogeneidad del grupo de control. Su limitación es que tiene mayor varianza cuando el tratado es un agregado de muchas unidades distintas, como ocurre en nuestro caso. SDiD combina lo mejor de los dos métodos anteriores, y por eso es nuestro principal.

■ Método 3: SDiD [1:50]

Diferencias en Diferencias Sintética, de Arkhangelsky y coautores, asigna pesos tanto a las unidades como a los períodos. Hereda la flexibilidad del Control Sintético y la simplicidad de Diferencias en Diferencias.

Lo elegimos como método principal por tres razones. Primero, es asintóticamente normal cuando hay muchas unidades tratadas, y nosotros tenemos 130. Segundo, es robusto a violaciones del supuesto de tendencias paralelas, que era justamente el punto débil del primer método. Y tercero, combina las ventajas de los otros dos enfoques en un solo estimador. Veamos cómo hacemos inferencia y qué pruebas de robustez aplicamos.

■ Inferencia y pruebas de robustez

[1:40]

La inferencia la hacemos por *bootstrap*, remuestreando unidades: 100 replicaciones para Diferencias en Diferencias y SDiD, y 2 mil para Control Sintético, que tiene mayor varianza por iteración.

Además, aplicamos tres pruebas de robustez que veremos en detalle más adelante. La primera incorpora variables de control del Banco Mundial. La segunda restringe el grupo de control excluyendo a los seis censos más severos. Y la tercera, la más exigente, combina placebos in-space con un análisis de sensibilidad leave-one-out al conjunto de donantes. Con el método ya planteado, pasemos a los resultados.

8. Resultados principales

(33:00–39:00) — Expositor 2

■ Resultados: ATT por método (Cuadro 3)

[1:40]

Este es el cuadro central de resultados. Les pido concentrarse en la última columna, la del método SDiD. Nueve de los diez lenguajes muestran un efecto positivo y estadísticamente significativo.

Los efectos más grandes están en JavaScript, con un aumento de 8,3 *unique pushers* por cada 100 mil habitantes; Python, con 4,5; y TypeScript, con 3,1. El único lenguaje sin efecto significativo es C#. Como referencia, las otras dos columnas muestran los resultados de Diferencias en Diferencias y de Control Sintético, que coinciden en el signo para los lenguajes principales. Resumamos los hallazgos.

■ Hallazgos clave

[1:00]

Tres mensajes principales. Primero, nueve de diez lenguajes presentan un efecto positivo y significativo. Segundo, las magnitudes más grandes están en los lenguajes con mayor presencia en la web y en ciencia de datos. Y tercero, eso es exactamente lo que predecía nuestra segunda hipótesis.

Ahora bien, hay que ser cuidadosos: la significancia por *bootstrap* no es la única evidencia que conviene revisar. Las pruebas de placebo refinan la lectura, y a ellas llegaremos en un momento. Antes, veamos el efecto de forma visual.

■ Tendencias y contrafactual: Python

[0:45]

En azul vemos la trayectoria del grupo tratado y en rojo la del control sintético construido con SDiD. Las dos trayectorias coinciden hasta el cuarto trimestre de 2022 y, a partir de ahí, divergen. Esa brecha es el efecto estimado, de 4,5 para Python. La coincidencia previa al tratamiento es lo que le da credibilidad al contrafactual.

■ Pesos del control sintético: Python

[0:45]

Aquí se ven los pesos del control sintético. Los puntos azules son los países donantes con un peso relevante. Lo importante es que el contrafactual de Python no depende de un solo país, sino de un conjunto diversificado de donantes, lo que lo hace más robusto. Sin embargo, no todos los lenguajes se comportan igual; JavaScript merece una advertencia.

■ Tendencias y contrafactual: JavaScript

[0:50]

En el caso de JavaScript se observa una ligera tendencia ascendente ya antes del tratamiento. Aunque el efecto es altamente significativo, esta pre-tendencia obliga a interpretarlo con cautela. Lo señalamos de forma explícita; y en las diapositivas de respaldo mostramos que se trata de un problema específico de este lenguaje, no del método, porque en otros lenguajes la pre-tendencia es plana. Esa honestidad nos lleva directamente a las pruebas de robustez.

9. Pruebas de robustez

(39:00–44:30) — Expositor 2

■ Robustez (1): control restringido

[1:00]

La primera prueba responde a una preocupación natural: ¿y si los resultados los estuvieran empujando los países más atípicos del control, como China, Rusia o Irán? Para verificarlo, reestimamos excluyendo a los seis censores más severos. Como muestra la tabla, las magnitudes son prácticamente idénticas a las originales. Es decir, el resultado no depende de esos casos extremos.

■ Robustez (2): placebos in-space y LOO

[1:30]

La segunda prueba es más exigente y tiene dos partes. Los placebos in-space asignan un tratamiento falso a cada uno de los países de control y reestiman el efecto; así obtenemos una distribución de efectos que no deberían existir, y comparamos nuestro efecto real contra ella. De ahí sale el p-valor placebo. El análisis leave-one-out, por su parte, reestima el modelo quitando un país de control a la vez, para verificar que ningún país individual esté conduciendo el resultado. Juntas, estas pruebas separan los efectos sólidos de los frágiles, y nos permiten clasificar la evidencia en tres niveles.

■ Tres niveles de evidencia

[1:30]

Según los placebos, clasificamos la evidencia así. Evidencia fuerte, con p-valor menor o igual a 0,05: Go y Ruby. Evidencia moderada, con p-valor menor o igual a 0,10: JavaScript, Python y TypeScript, que son los tres lenguajes priorizados por nuestra hipótesis. Y evidencia débil: C, C++, Java, PHP y C#.

Quiero ser explícito en algo: en los lenguajes de evidencia débil, la significancia por *bootstrap* convive con p-valores placebo que no la respaldan. Ese contraste sugiere que hay heterogeneidad estructural del grupo de control que el contrafactual no absorbe del todo. Lo decimos abiertamente. Lo resumimos en dos figuras.

■ Forest plot

[0:45]

Este *forest plot* resume todo en una sola imagen. El punto rojo es el efecto observado; la barra azul es el rango del leave-one-out; y la banda gris es el intervalo placebo al 95 por ciento. La lectura es directa: donde el punto rojo cae fuera de la banda gris, el efecto es robusto. La distribución placebo lo muestra lenguaje por lenguaje.

■ Distribución placebo

[0:45]

Para cuatro lenguajes representativos, el histograma gris muestra los efectos placebo y la línea roja el efecto real que estimamos. Cuanto más a la derecha de la masa gris cae la línea roja, más creíble es el efecto. Con esto cerramos la parte de robustez y pasamos a las conclusiones.

10. Conclusiones y recomendaciones

(44:30–49:30) — Expositor 2

■ Limitaciones reconocidas

[1:15]

Reconocemos cinco limitaciones. Primera, la asignación al control no es aleatoria: los países sin ChatGPT son geopolíticamente distintos. Segunda, una parte de los desarrolladores en países bloqueados pudo acceder por VPN, lo que atenúa el efecto hacia cero. Tercera, la ventana postratamiento es corta, de solo cuatro trimestres. Cuarta, la variable mide cantidad, no calidad ni intensidad. Y quinta, la pre-tendencia de JavaScript que ya mencionamos. Un punto a favor: varios de estos sesgos juegan en contra de encontrar un efecto, de modo que nuestro estimado es probablemente conservador, un piso del efecto real.

■ Conclusiones (1): hallazgo central

[1:15]

Nuestra conclusión central es que la disponibilidad de ChatGPT incrementó la actividad de desarrollo de software en los países con acceso, con un efecto positivo y robusto en los lenguajes que la teoría priorizó: JavaScript, Python y TypeScript.

Este hallazgo es consistente con la teoría: con Acemoglu y Restrepo, por la reasignación de tareas hacia los segmentos de menor productividad previa; y con Stigler, por el shock negativo sobre los costos de búsqueda. Además, los tres métodos econométricos coinciden en el signo, y las pruebas de placebo sostienen la interpretación causal en el subconjunto robusto.

■ Conclusiones (2): heterogeneidad

[0:50]

El efecto no es uniforme entre lenguajes; se gradúa según la fuerza de la evidencia. Por eso fundamentamos nuestras recomendaciones de política únicamente en el subconjunto con evidencia fuerte o moderada, y no sobreinterpretamos los lenguajes con evidencia débil.

■ Recomendaciones de política

[0:50]

De aquí derivan dos recomendaciones concretas. La primera es un acceso subsidiado a ChatGPT Plus para desarrolladores junior orientados a ciencia de datos, por períodos iniciales renovables. La segunda es un sistema de bonos por productividad asistida por inteligencia artificial. Ambas se alinean con el mecanismo de Acemoglu y Restrepo: la herramienta ayuda más a los segmentos del mercado con menor productividad previa.

■ Contribución y futuras investigaciones

[0:50]

Resumiendo nuestra contribución: este es el primer estudio que estima el efecto causal de ChatGPT sobre la actividad de desarrollo de software desagregada por lenguaje de programación, aplicando el estimador SDiD sobre 130 unidades tratadas y con un conjunto completo de pruebas de robustez. Hacia adelante, proponemos extender la ventana postratamiento hasta 2025 y 2026, modelar de forma explícita el mecanismo, y explorar otros resultados como commits o repositorios creados.

11. Cierre

(49:30–50:00) — Ambos

■ Gracias

[0:30]

En síntesis, la disponibilidad de ChatGPT tuvo un efecto causal positivo y heterogéneo sobre el desarrollo de software, más fuerte en los lenguajes que la teoría predijo. Agradecemos al jurado por su atención y quedamos atentos a sus consultas y comentarios.

12. Respuestas a preguntas frecuentes

(solo si las hacen)

Lo siguiente no forma parte de la exposición; son respuestas para decir en voz alta solo si el jurado pregunta. A la derecha de cada pregunta se indica quién la responde, según el área que le corresponde; el slide de respaldo a abrir va al final de cada respuesta.

■ Si preguntan: ¿el grupo de control no es demasiado distinto?

[Expositor 2]

Es una preocupación válida. Por eso corrimos una prueba que excluye a los seis censores más severos, y los resultados casi no cambian. Además, esa diferencia juega en contra de encontrar un efecto, así que nuestro estimado es conservador. (*Abrir A.1 y Robustez 1.*)

■ Si preguntan: ¿y el acceso por VPN?

[Expositor 2]

El acceso por VPN en países bloqueados atenúa el efecto hacia cero, porque contamina al grupo de control con unidades parcialmente tratadas. Eso significa que el efecto real podría ser incluso mayor que el que reportamos; nuestro estimado es un piso. (*Slide de Limitaciones.*)

■ Si preguntan: ¿cómo descartan pre-tendencias?

[Expositor 2]

Con los análisis de evento. En Python y TypeScript la pre-tendencia es plana, lo que valida el supuesto. En JavaScript sí hay una leve pre-tendencia, pero es un problema específico de ese lenguaje, no del método, como muestran los otros casos. (*Abrir A.3, A.4 y A.5.*)

■ Si preguntan: ¿por qué SDiD y no solo DiD?

[Expositor 2]

Porque Diferencias en Diferencias exige tendencias paralelas, un supuesto dudoso con nuestro control heterogéneo. SDiD es robusto a esas violaciones y es asintóticamente normal con nuestras 130 unidades tratadas. (*Slide del Método 3.*)

■ Si preguntan: ¿los controles cambian los resultados?

[Expositor 2]

No. Al incorporar las variables de control del Banco Mundial, uso de computadora y de internet, los efectos se mantienen e incluso se fortalecen. (*Abrir A.2.*)

■ Si preguntan: ¿de dónde sale la ecuación estimable?

[Expositor 1]

De un modelo de tareas que derivamos en ocho pasos. El coeficiente del tratamiento se factoriza en la exposición del lenguaje y la absorción del país. (*Abrir Apéndice B.*)

■ Si preguntan: ¿por qué C# no da efecto?

[Expositor 2]

Por una menor exposición relativa del lenguaje y una evidencia placebo débil. Preferimos reportarlo así, sin forzar una interpretación que los datos no sostienen. (*Slide de Tres niveles de evidencia.*)

■ Si preguntan: ¿esto es causalidad o correlación?

[Ambos]

Es causalidad, dentro de los límites del diseño. Tenemos un diseño cuasi-experimental, tres métodos que coinciden, placebos in-space y leave-one-out. La evidencia causal es robusta en el subconjunto principal de lenguajes. (*Abrir A.1 y Robustez 2.*)